ML system design interview

Harmful content detection on a social platform

LECTURE 9 MOCK INTERVIEW

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Eight lectures built every piece a production system needs. Today they are put together under interview conditions.

The piece	Where we saw it
Framing a task, a baseline to beat, the end-to-end pipeline	Lectures 1–2
Precision, recall, thresholds as decisions, honest splits	Lecture 3
Probabilities, log loss, calibration	Lecture 4
Trees and ensembles on tabular data; rare positives and resampling	Lecture 5
Clustering, anomalies, where to spend a few labels	Lecture 6
Networks, output layers and losses, a training loop in PyTorch	Lectures 7–8

✓ Nothing new is needed to design a real system. What is new is doing it out loud, in order, with trade-offs.

Today

1. **The format** — what an ML system design interview is, and how it is scored
2. **The prompt** — detect harmful content on a social platform
3. **Nine steps** — at each one you answer first, in pairs; then a strong answer, and the red flags an interviewer writes down
4. **The whole system** on one slide, the follow-up questions, and how to practise

There is no notebook today: an interview is a whiteboard and a conversation.

What an ML system design interview is

- A standard round for machine learning engineers and applied scientists at Meta, Google, Amazon, Microsoft, Apple and most companies that ship models
- One open-ended prompt — “design the system that …” — and a whiteboard. There is no single right answer
- **The candidate drives.** The interviewer plays the product manager: answers questions, adds constraints, and pushes on whatever looks weak
- It tests whether you can turn a vague business problem into a system that works, can be measured, and can be run

> **WHAT IT IS NOT**
>
> A quiz on the newest architecture. Candidates who open with “I would fine-tune a large model” are usually marked down: they have answered a question nobody asked yet.

How it is scored

Signal	A weak answer	A strong answer
Framing	“Train a classifier”	asks what the output is used for, and what each mistake costs
Metrics	accuracy, or AUC alone	the metric the decision needs, offline and online
Data	assumes clean labels exist	says where labels come from, and how they are biased
Modelling	names the biggest model	a baseline first, and a reason for every step up
Production	stops at training	serving, cost, monitoring, feedback loops
Communication	a list of techniques	trade-offs: “X, because Y; Z would cost W”

✓ Every row is something this course has measured. The interview checks that you reach for it without being asked.

The shape of a good answer

1. **Clarify** the problem and its constraints
2. **Frame** it as a machine learning task: input, output, decision
3. **Metrics**, offline and online
4. **Data and labels**
5. **Features**
6. **The model**, from baseline up
7. **Training, thresholds and offline evaluation**
8. **Serving** at scale
9. **Launch, monitoring and iteration**

✓ Cover all nine at the level of a sentence each before going deep anywhere; then go deep where the interviewer pulls. Running out of time at step four is the most common way to fail.

How we run it today

- I am the interviewer. You are the candidate, in pairs
- At every **Your turn** slide: write your answer down before the next slide — a sentence or a sketch is enough
- Then a strong answer, and the red flags an interviewer would note
- Say your answers as trade-offs, and say your assumptions aloud: an interviewer can only credit reasoning they can hear

> **THE RULE OF THE ROOM**
>
> Wrong answers are the point. The strong answer is one good answer, not the only one; if yours differs, argue for it.

THE INTERVIEWER

“We run a social platform with billions of users. Design a system that detects harmful content.”

What do you say first?

Step 1 · Your turn

Before you design anything: what do you ask?

Write down the questions you would ask the interviewer before drawing a single box.

Think about what is being detected, in what, how much of it, how fast, what happens next, and what you can learn from.

Step 1 — The questions a strong candidate asks

Question	Why it changes the design
Which harms? Violence, nudity, hate speech, harassment, self-harm, scams…	each policy is a label, a threshold and a team
Which content? Text, images, video, comments, live streams	the modality decides the features and the cost
How much, and in how many languages?	the cost of every prediction, and where the model is blind
Before publishing, or after?	the latency budget
What happens to flagged content?	the output is an action, not a label
What labels exist today?	everything downstream depends on it
Appeals, regulators, transparency reports?	what must be explained and measured

Step 1 — What the interviewer answers

- **Harms:** violence, nudity, hate speech, harassment and self-harm, each with its own written policy
- **Content:** posts with text and images; hundreds of millions a day, in dozens of languages
- **When:** check at upload within tens of milliseconds, and again as a post spreads
- **Actions:** remove automatically when sure, send to a human reviewer when unsure, reduce distribution in between
- **Labels:** past reviewer decisions and user reports; users can appeal; a regulator expects a public report on what was removed

> **RED FLAG**
>
> Starting to design before asking. Second red flag: asking twenty questions and never committing. Ask the few that change the design, write the answers on the board as assumptions, and move on.

Step 2 · Your turn

State the machine learning task

What goes in, what comes out, and what is the output used for? One model or several?

Step 2 — Input, output, decision

- **Input:** a post — its text and image, its author, where and when it was posted
- **Output:** one probability per harm, $p(\text{violence} \mid \text{post}), p(\text{hate} \mid \text{post}), \ldots$ — *multi-label*: a post can be violent and hateful at once
- **Decision:** a separate *policy layer* turns the probabilities into actions with a threshold per harm and per action

Post → model → one probability per harm → policy layer → remove, send to review, reduce distribution, or leave

> **RED FLAG**
>
> One softmax over the harms. A softmax makes the probabilities sum to one, so a post cannot be both violent and hateful; use one sigmoid output per harm. And keep the action out of the model: policies change weekly, weights should not have to.

Step 2 — One model, one per harm, or one with many heads?

Option	For	Against
One binary model, harmful or not	simplest to build	policies differ in cost; one threshold cannot serve them; the action cannot say why
One model per harm	independent teams and releases	many models to serve; rare harms have too few labels to learn alone
One shared model, one head per harm	**rare harms borrow from common ones; one pass per post**	**retraining couples the harms; one head's change must be checked on all**

✓ A strong candidate picks one and says what it costs: "a shared model with a head per harm, because the rare harms need the shared representation; I will re-evaluate every head on every release."

Step 3 · Your turn

How do you know it works?

Name the metrics you would report offline, before launch, and online, after it.

Harmful posts are a tiny fraction of everything posted. What does that do to accuracy?

Step 3 — Offline: precision and recall, per harm

- Violating posts are a fraction of a percent of all posts, so “nothing is harmful” is right almost every time: **accuracy is meaningless**
- Per harm, the **precision–recall curve**; the area under it rather than under the ROC curve, which looks excellent when negatives dominate
- Read it at the operating points the actions need: recall at the precision required to *remove automatically*, and recall at the precision reviewers can afford
- **Calibration**, because the policy layer and the queue size both use the probabilities as probabilities

> **RED FLAG**
>
> One number for all harms. Missing self-harm and missing spam do not cost the same; report each harm, and say which one you would trade for which.

Step 3 — Online: what users actually see

Metric	What it measures
Prevalence of each harm	the share of *views* that land on violating content, estimated by having reviewers label a random sample of what was viewed
Proactive rate	the share of removed content caught before any user reported it
Appeals overturned	wrong removals: a precision signal from the field
Reviewer queue	how much reaches humans, and how long it waits
Guardrails	creators' complaints, engagement, time to action

RED FLAG

"Number of posts removed" as the goal. It rewards removing everything; prevalence weighted by views is what the platform owes its users.

Step 4 · Your turn

Where do the labels come from — and what is wrong with them?

List the sources of labels you can use, and one problem with each.

Step 4 — The labels, and their biases

Source	Good	Wrong with it
Reviewer decisions	trained people, written policy	expensive; exist only for what was flagged — *selection bias*
User reports	cheap, plentiful	noisy; mass-reporting campaigns; popular posts get reported more
Appeal outcomes	the hardest cases, judged twice	few; only from users who bother to appeal
Known-bad matches	exact copies of known images	catches only what has been seen before

- Reviewers disagree: **measure agreement** on a doubly labelled slice; it is the ceiling for any model
- Policies change, and a label is only valid for the policy it was given under: **store the policy version with every label**

Step 4 — Spend labels well, and split honestly

- **A random sample** of all traffic, labelled every week: the only unbiased estimate of prevalence and of what the model misses
- **Targeted labelling** for training: the posts the model is least sure about, and representatives of new clusters — where a few labels buy the most, as with Lecture 6's forty
- **Split by time**: train on the past, validate and test on later weeks, as the system will meet them
- **Split by author** and remove near-duplicate reposts across the split

> **RED FLAG**
>
> A random split of posts. The same author's style and the same viral image land on both sides; the model memorises them and the offline score is a memory test.

Step 5 · Your turn

What does the model look at?

Group the features you would use, and say which ones exist at the moment of upload.

Step 5 — Features

Group	Examples	Available at upload?
Text	a pretrained multilingual text encoder's vector of the caption — opened later in the course; today a black box	**yes ✓**
Image	a pretrained image encoder's vector; text read out of the image	**yes ✓**
Author	account age, past violations, followers, verified	**yes ✓**
Context	where it is posted, the group, the language, the hour	**yes ✓**
Reaction	reports, shares, how fast it spreads, the comments	**only later ✗**

RED FLAG — LEAKAGE IN TIME

Training on "number of reports" and serving at upload, where it is always zero. Compute every feature as of the moment of the decision; use an upload-time model, and re-score with reaction features as they arrive.

Step 5 — The data fights back

- Bad actors adapt: misspellings, characters swapped for look-alikes, hateful text written *inside* an image, coded words that change every month
- Features that survive this: characters rather than whole words, text read out of images, the author's history, the network of accounts posting the same thing
- A benign image with a hateful caption is harmful only in combination: the model must see both together

✓ Say this unprompted. It is what separates a candidate who has run a system from one who has trained a model.

Step 6 · Your turn

Which model — and how do you get there?

Start from the simplest thing that could work, and say what would make you move up each step.

Step 6 — From baseline to the shipped model

Stage	Model	Why
Baseline	rules: lists of banned terms, matching against known-bad images	ships in days, explainable, the bar to beat
First model	logistic regression on the frozen vectors and the tabular features	fast, calibrated, a real metric to improve on
Tabular strength	gradient-boosted trees on the same features	interactions between author, context and content
Shipped	**a network: the vectors and features together → shared layers → one sigmoid head per harm**	**sees text and image together; rare harms share what common ones learn**
Maybe later	fine-tuning the encoders themselves	only if the offline gain pays for the serving cost

✓ Every step up is justified by the offline metric of step three, measured on the split of step four.

Step 6 — The shipped model, in detail

- **Early fusion:** concatenate the text vector, the image vector and the tabular features before the shared layers, so a benign picture and a hateful caption are judged together. Late fusion — one classifier per modality, scores combined — misses exactly that case
- **Loss:** the sum over harms of binary cross-entropy, one term per head, weighted so the rare harms are not drowned out
- **Imbalance:** keep every positive, sample the negatives, then correct the probabilities back — the log-odds shift of Lecture 5's advertising design exercise

> **RED FLAG**
>
> Class weights or resampling with no correction, then thresholds set as if the outputs were probabilities. The policy layer needs calibrated probabilities; check them per harm, as in Lecture 4.

Step 7 · Your turn

Before launch: how do you set the thresholds?

Each harm needs a rule for “remove”, “send to a human” and “reduce distribution”. Who decides, and from what?

Step 7 — Thresholds are policy, set from the curves

Band of $p(\text{harm})$	Action	Set by
very high	remove automatically, notify, allow appeal	the precision the policy team accepts for no human in the loop
high	send to a human reviewer	reviewer capacity: the expected daily volume must fit the queue
medium	reduce distribution while it is reviewed	the cost of spreading a harmful post against hiding a good one
low	leave it	—

Each band is a point on the validation precision–recall curve of that harm, chosen with the policy team and signed off — as Lecture 3 put it, the threshold is a decision, not a hyperparameter.

Step 7 — Evaluate where it fails, not on average

- Report every metric per language, per region, per content type, for new and for established accounts
- The known failure: dialects and reclaimed words flagged as hate speech far more often than standard usage, so one community's posts are removed more often for saying the same thing
- Report the **worst slice** beside the average, and set a floor that every slice must clear before launch
- Read a sample of false positives and false negatives by eye before any launch decision

> **RED FLAG**
>
> One global number. Lecture 10 is about exactly this: what a model relies on, and for whom it fails.

Step 8 · Your turn

Hundreds of millions of posts a day

Draw the path a post takes from upload to action. Where do you spend the expensive computation?

Step 8 — Serving: cheap first, expensive where it matters

- **Stage one, at upload, on everything:** known-bad matching and a small model, within the latency budget. Clearly fine: publish. Clearly violating: block
- **Stage two, in the background:** the full multimodal model on the uncertain posts, published meanwhile with reduced distribution
- **Re-score** as reaction features arrive and as a post starts to spread
- **The review queue** is ordered by expected harm: probability of violation times predicted reach — the post about to go viral first

✓ Compute each post's text and image vectors once, store them, and share them with the other systems that need them.

Step 8 — The trade-offs to say out loud

Decision	The trade-off
Check before or after publishing	safety against the delay every user feels
Size of the stage-one model	what it catches against what the latency budget allows
When the model is down	fail open (publish unchecked) or fail closed (hold every post): a policy choice, decided in advance
How much reaches humans	accuracy on the hard cases against cost — and against the reviewers' wellbeing, which limits how much they can see

Step 9 · Your turn

You have launched. How do you know it works — and keeps working?

How do you roll it out, what do you watch, and what makes it degrade?

Step 9 — Rolling it out

- **Shadow mode:** the new model scores live traffic but takes no action; compare its decisions with the reviewers'
- **Gradual ramp:** a growing share of traffic, watching prevalence, appeals overturned and the queue
- **A/B test** against the current model on prevalence and appeals — but a control group cannot be left exposed to the worst harms, so for those the random-sample prevalence estimate replaces a pure holdout

> **RED FLAG**
>
> Launching on an offline gain alone. The model changes what is shown, what users report and what reviewers see; only the live measurement counts.

Step 9 — What makes it degrade, and what to watch

Cause	What to watch, and what to do
New slang, new formats, news events	flag rates and input distributions per harm and language; relabel and retrain
Bad actors adapting	the random sample catches what the model now misses; red teams
The feedback loop	labels come only from what the model flagged, so it never learns its misses — keep labelling the random sample
A policy change	old labels become wrong; relabel under the new policy version

✓ Retrain on a schedule, but only release a model that beats the current one on the latest weeks and clears every slice.

The whole system, on one slide

Upload → stage one: known-bad matching and a small model → publish, block, or publish with reduced distribution → stage two: the multimodal model, one head per harm → policy layer: thresholds per harm and per action → remove, send to review (ordered by probability times reach), or leave → reviewers' decisions and appeals become labels → a random sample is labelled every week → retrain, test on the latest weeks and on every slice → shadow, ramp, A/B test.

Measured by	Offline	Online
Each harm	precision and recall at the action thresholds; calibration; worst slice	prevalence; proactive rate; appeals overturned; queue

The follow-up questions interviewers ask

“What if …”	The gist of a good answer
a new kind of harm appears overnight?	rules and reviewers first; a new head on the shared model; targeted labelling
you launch in a new language?	a multilingual encoder, labels in that language, the slice measured before launch
bad actors misspell every word?	character-level and image-text features, adversarial examples in training
reviewers are overwhelmed?	order the queue by reach; act automatically only where precision allows; never just lower the bar
a famous account is wrongly removed?	appeals and a second review; the same rules for every account

The red flags an interviewer writes down

- Designs before asking a single question — or asks and never commits
- Accuracy, or one number for everything
- Assumes labels are free, clean and unbiased
- A random split of posts; features that exist only after the decision
- Starts with the biggest model; cannot say what the baseline is
- A softmax over harms that can co-occur
- Stops at training: no serving path, no cost, no monitoring, no feedback loop
- Lists techniques instead of choosing one and saying what it costs

How to practise

- Take any product you use and run the nine steps on it, out loud, against a clock
- Say every choice as a trade-off; write assumptions on the board
- Prompts in the same style: video recommendations for a streaming app; the arrival time shown by a food-delivery app; ranking an e-commerce site's search results; blocking fraudulent payments at checkout; "people you may know"
- Every lecture from here on ends with a design exercise in this format: it is practice for this conversation

What we did

1. Ran a mock ML system design interview, in the format used for machine learning engineers at the large technology companies: one open prompt, the candidate drives, trade-offs out loud
2. Clarified before designing, and framed the task as one probability per harm — multi-label, sigmoid heads — with a separate policy layer that turns probabilities into actions
3. Measured with precision and recall per harm at the action thresholds, and online with prevalence weighted by views, never accuracy
4. Treated labels as biased evidence: random-sample labelling, targeted labelling, splits by time and by author, policy versions
5. Built up from rules to a fused multimodal network, set thresholds as policy, evaluated per slice, served in two stages, and launched through shadow mode and a ramp, watching for drift, adversaries and the feedback loop

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 9

five, in the style of the written paper

Exercises — Lecture 9

1 A harmful-content model ends in one softmax over five harms. Say why this is the wrong output layer, and what replaces it. [4]

2 Harmful posts are a fraction of a percent of all posts. Explain why accuracy and the area under the ROC curve both mislead here, and say what you would report offline for each harm. [5]

3 The training labels are past reviewer decisions on posts the current system flagged. Name the bias, its consequence for the next model, and how to correct for it. [5]

Solutions on Lecture 10’s deck.

Exercises — Lecture 9 (continued)

4 Adding “number of user reports” sharply raises the offline recall of the model that checks posts at upload. Explain the gain, and say what you would do. [4]

5 For each harm the policy layer has three thresholds: remove automatically, send to a human reviewer, reduce distribution. Say what sets each one. [5]

Solutions on Lecture 10’s deck.

THE FIVE SET AT THE END OF LECTURE 8

Solutions Lecture 8

not part of this lecture

Solutions — Lecture 8

1. Reverse-mode automatic differentiation costs one sweep for all partial derivatives; forward mode costs one…

✓ Many inputs, one output — which is exactly a loss.

- Forward mode's cost scales with the number of inputs, reverse mode's with the number of outputs
- A neural network has millions of parameters and one scalar loss

2. Test accuracy on 10,000 images is computed as the plain mean of the per-batch accuracies, in batches of 384.…

✓ The last batch holds 16 images but is weighted as heavily as a full batch of 384. Count the hits over the whole set, or weight each batch by its size.

- 10,000 = 26 × 384 + 16, so there are 27 batches and the last is short
- Measured, the error was 0.067 points — small here, but you cannot tell from the number whether it was

Solutions — Lecture 8 (2)

3. The five lines of the training loop are reordered so that opt.step() comes before loss.backward(). State what…

✓ Nothing raises, and the training loss stays at its starting level: the parameters are never changed.

- The step is taken with the gradients zeroed at the top of the iteration
- The gradients computed afterwards are cleared before they are ever used

4. A network with dropout is evaluated without calling model.eval(). Describe the symptom precisely, and the…

✓ The score wobbles between identical calls. Evaluate twice and assert the two agree.

- Dropout keeps sampling masks in training mode, so the metric is a random variable
- A deterministic function of fixed weights and fixed data does not change between calls

Solutions — Lecture 8 (3)

5. In the measured comparison the accelerator barely beat the CPU on the smallest network, and its lead grew as…

✓ Dispatch overhead: on a small matrix product much of each step is the cost of dispatching it, not the arithmetic. The crossover is set by how much arithmetic each batch carries — measured, near-parity at the smallest width and a clear lead at the widest.

- A GPU is a machine for large matrix products, and a narrow layer at a batch of 128 is not a large one
- The device pulls ahead once each batch carries enough arithmetic to cover the dispatch, and that grows with the layer widths

LECTURE 10 · OUTSIDE THE BOOK

Explaining and auditing models

What a model relies on: permutation importance, partial dependence, Shapley values

For whom it fails: slices, fairness criteria, proxies

The audit a regulator will ask for